
Iconic (.ico Utilities)

Release 1.0.0

Doji Technologies

Oct 28, 2022

OVERVIEW

1	Loading .ico files	2
2	Encoding .ico files	3
2.1	Fire & Forget	3
2.2	Customizing Multi-Resolution Behaviour	3
3	FAQ	6
3.1	Why not use System.Drawing.dll?	6

Iconic is a cross-platform utility for reading and writing .ico files. It doesn't rely on System.Drawing.dll and is suited to be used on all platforms that Unity supports.

Feel free to check out [this WebGL Demo](#), showcasing what can be achieved with this asset.

Load .ico files at runtime Extract all resolutions contained in a single .ico file into Texture2D objects Supports transparency

Write .ico files at runtime Convert Texture2D objects to .ico file automatically create multi-resolution icons

LOADING .ICO FILES

Usage of the utility is pretty simple. There is a static class called 'IconConversion' that you can use to access various methods. You can load an icon from various sources.

E.g. you can use one of the following methods

```
IconConversion.LoadIcon(string fileName)
IconConversion.LoadIcon(Stream stream)
IconConversion.LoadIcon(byte[] array)
```

Assuming we load an icon from a byte array 'myIconData', we'll use:

```
Icon icon = IconConversion.LoadIcon(myIconData);
```

Then we can use an extension method to turn the icon into a Texture2D like so

```
Texture2D myTexture = icon.ExtractTexture2D(imageIndex: 0);
```

As you see, you need to pass the index of the image to this method. An .ico file can contain multiple images, usually each with a different resolution. You can check `Icon.NumImages` for the total number of images that an icon contains.

ENCODING .ICO FILES

2.1 Fire & Forget

It only takes one simple method call to generate an .ico file from a Texture2D:

```
IcoConversion.EncodeToICO(myTexture);
```

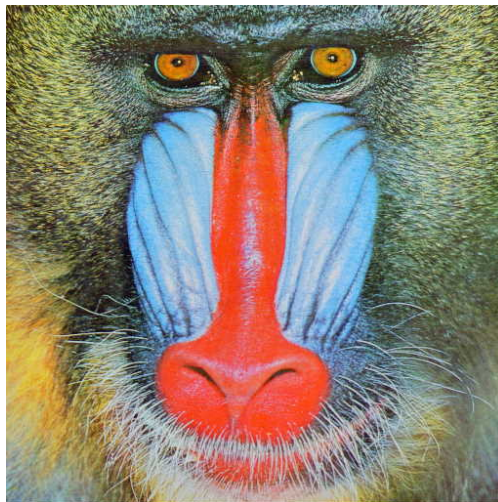
Some prerequisites for the textures:

- must be in a non-compressed format (RGBA32, ARGB32, RGB24)
- .ico only supports resolutions up to 256x256

2.2 Customizing Multi-Resolution Behaviour

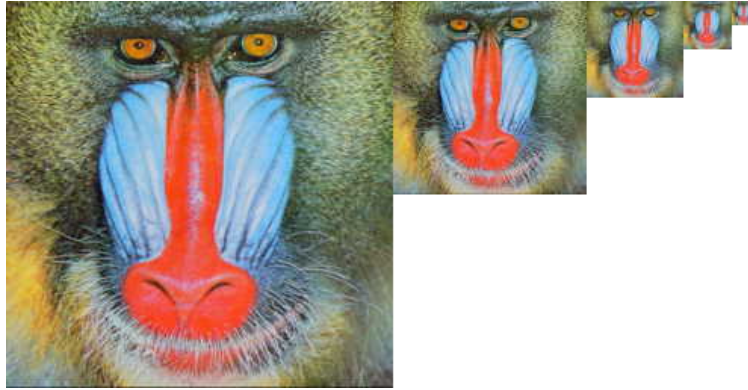
By default, the `EncodeToICO()` methods automatically create a multi-resolution .ico file. (Visual Studio is a decent tool to inspect .ico files by the way. If you want to know which images are embedded in an .ico file just drag and drop it into an open instance of VS.)

Configuring this behaviour can be achieved by passing an `IcoRescaleSettings` object. This is best explained with a few examples. Imagine we have the following image with a resolution of 200x200 as an uncompressed Texture2D object in Unity:



2.2.1 Example 1: Default Values

If you just call `IcoConversion.EncodeToICO(myTexture);` without specifying any settings, the resulting .ico file will contain the following 5 images with the resolutions 256x256, 128x128, 64x64, 32x32 and 16x16:



2.2.2 Example 2: One Single Resolution

If you do not wish to create a multi-resolution icon you can simply set `MultiResolutionIcon` to false:

```
1 settings = new IcoRescaleSettings() {  
2     MultiResolutionIcon = false  
3 };
```

This will result in an .ico file that contains one single 200x200 image.

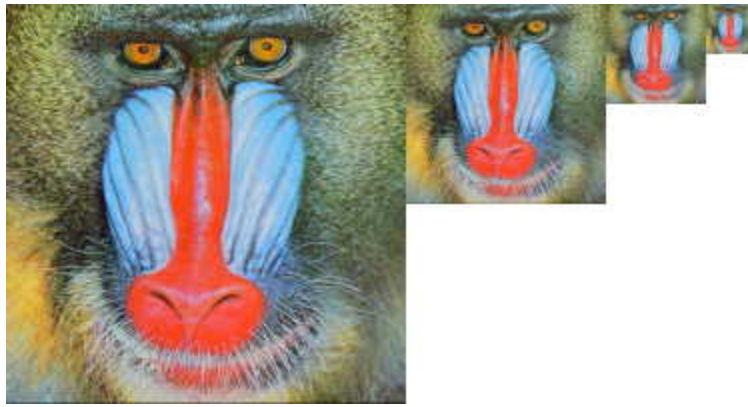
Note: When `MultiResolution` is set to false, the asset will attempt to turn your texture into an .ico file without modifications. This means, you have to ensure yourself, that your texture is within the maximum resolution supported by .ico files (256x256). Otherwise you will encounter a runtime exception like: "Invalid texture resolution. ICO files support a maximum resolution of 256x256."

2.2.3 Example 3: Different Scaling Modes

In example 1 you might have noticed, that the largest image used a resolution of 256x256 even though the initial texture has a resolution of 200x200. The reason is, that `IcoRescaleSettings.DownscalesMode` is set to 'DownscaleMode.ToNearest' by default. Thus the texture will be rescaled to the nearest power-of-two. If you wish to change this behaviour, you can specify a different mode like so:

```
1 settings = new IcoRescaleSettings() {  
2     MultiResolutionIcon = true,  
3     DownscaleMode = DownscaleMode.None  
4 };
```

When specifying 'DownscaleMode.None', the resulting .ico file will contain the following 4 images with the resolutions 200x200, 100x100, 50x50, 25x25:



The other modes 'DownscaleMode.ToLarger' and 'DownscaleMode.ToSmaller' work similar to the default 'DownscaleMode.ToNearest', but you can choose to always scale up or down to the next power-of-two respectively.

Note:

`IcoRescaleSettings.DownscalesMode` only has an effect if `IcoRescaleSettings.MultiResolutionIcon` is set to true. Irrespective of which downscale mode is used, the largest image will always be clamped to the maximum supported resolution of 256x256 pixels.

3.1 Why not use System.Drawing.dll?

System.Drawing contains platform-specific code, which means your code will only run on Windows platforms. Iconic can be used from all platforms that Unity supports.